ECE 5780/6780 Lab 1 – Spring 2025

Due Date: Friday, January 17 (20 points)

Objectives

The purpose of this lab is to become familiar with the process of creating a multi-task microcontroller application using FreeRTOS, Keil µVision, and the STM32L476 Nucleo-64 Board.

Overview

In this lab you will create, download, execute, and debug a FreeRTOS multi-task C program that controls the LED on the STM32L476 Nucleo-64 Board using the on-board user push button.

Preparation

You will need your STM32L476 Nucleo-64 Board.

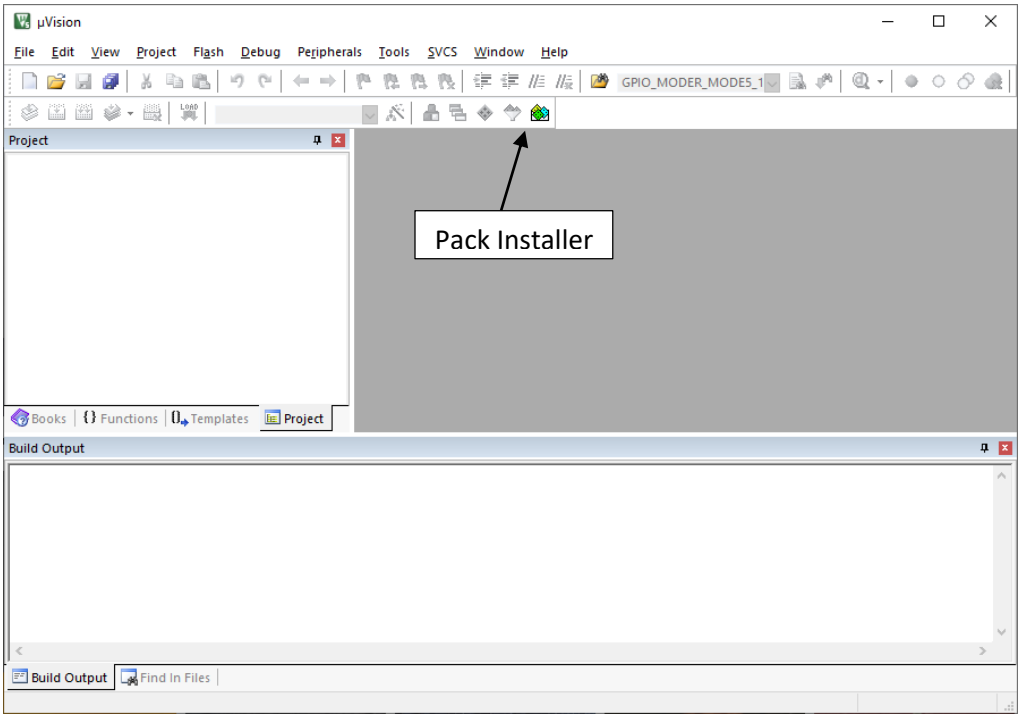
Procedure

1. You may need to install the ARM::CMSIS-FreeRTOS pack in Keil. If so, follow these steps.

   a. Start Keil µVision.

   b. Make sure there are no projects open.

   c. Start the **Keil Pack Installer**. See figure below.



   d. When the installer opens, it will run package updates. This could take several minutes, depending on the number of updates needed. Wait until updates are complete (See status bar at bottom of window).

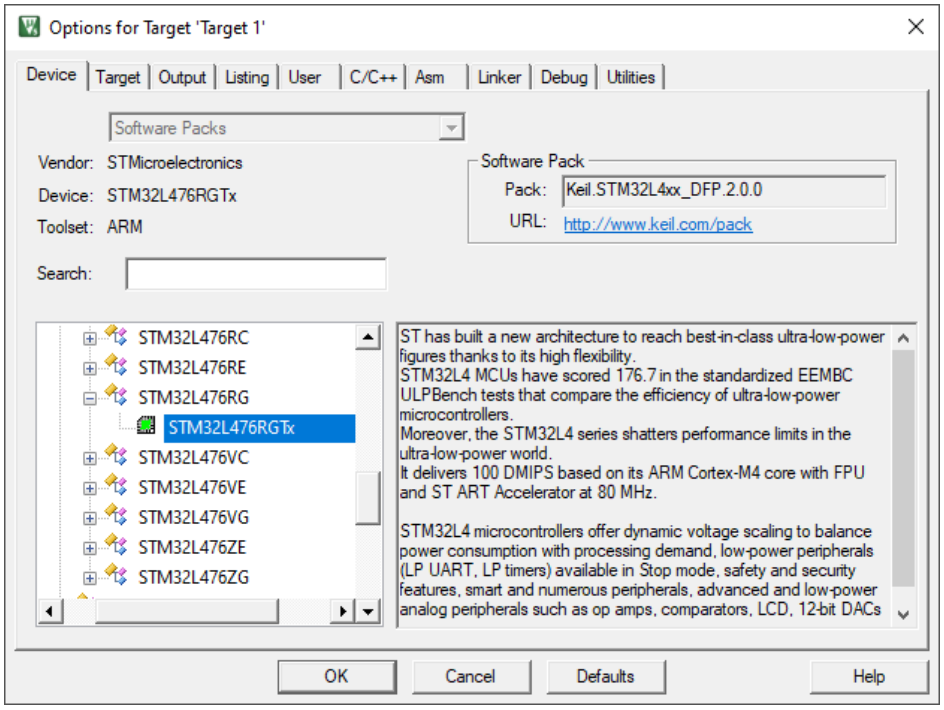
   e. In the right-side window of the Pack Installer, expand the Generic menu and look for **ARM::CMSIS-FreeRTOS**. If it is marked "Up to data" nothing else needs to be done. If it is marked "Install+", click on the button and wait for installation to complete.

2. Create a new Keil µVision project that uses FreeRTOS and targets the STM32L476 Nucleo-64 Board.

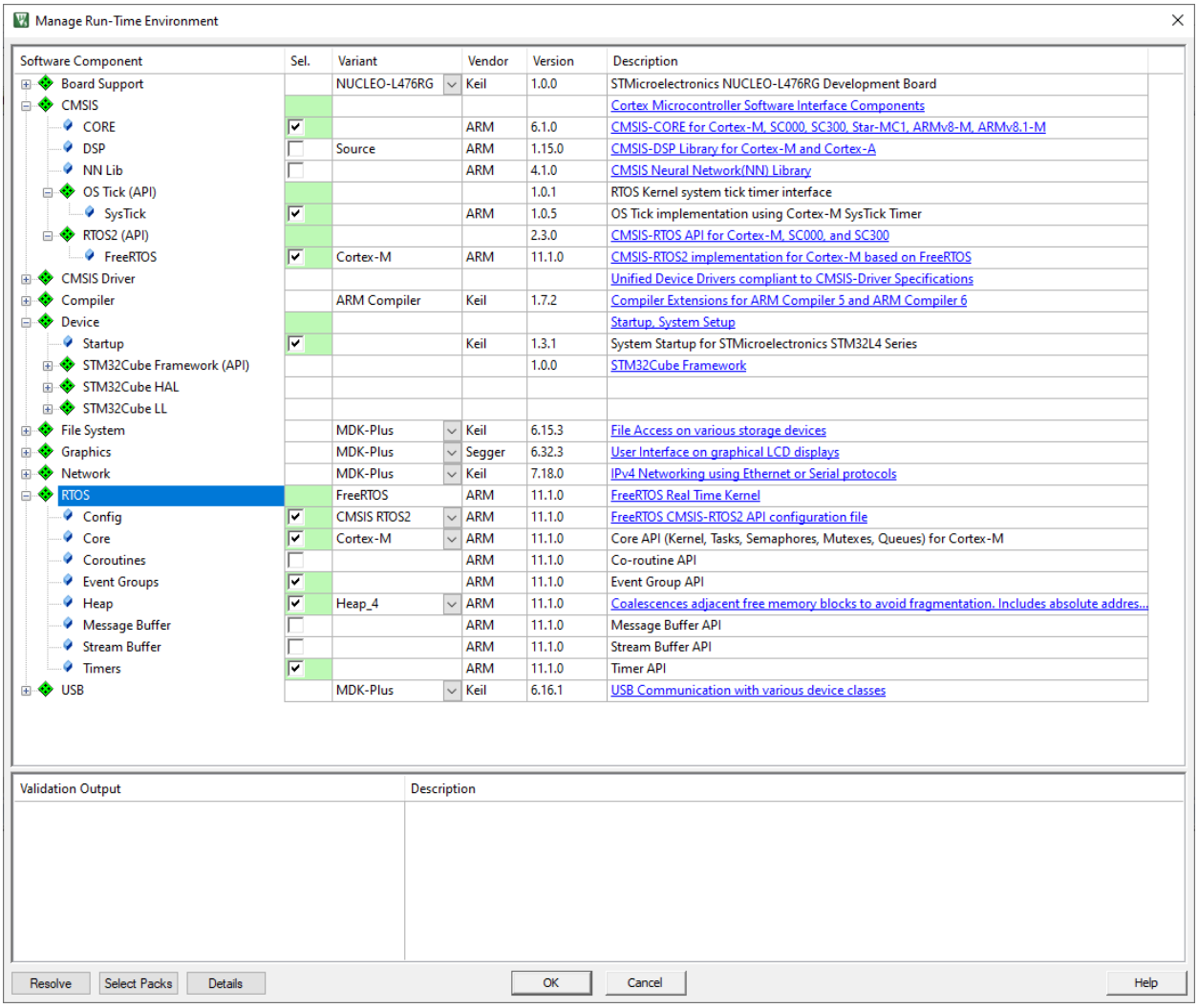
   a. If you skipped step 1, start Keil µVision.

   b. Select **Project -> New µVision Project** and give your project a name and a path that make sense to you.

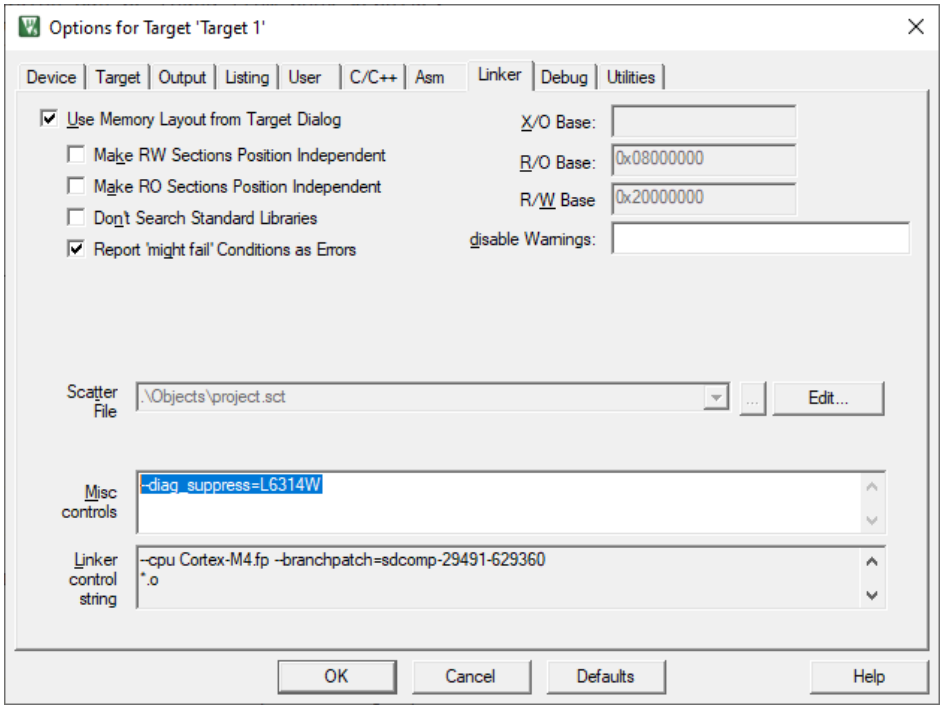
c. Select the appropriate microcontroller. Our Nucleo boards use the STMicroelectronics STM32L476RGTx. See the figure below.



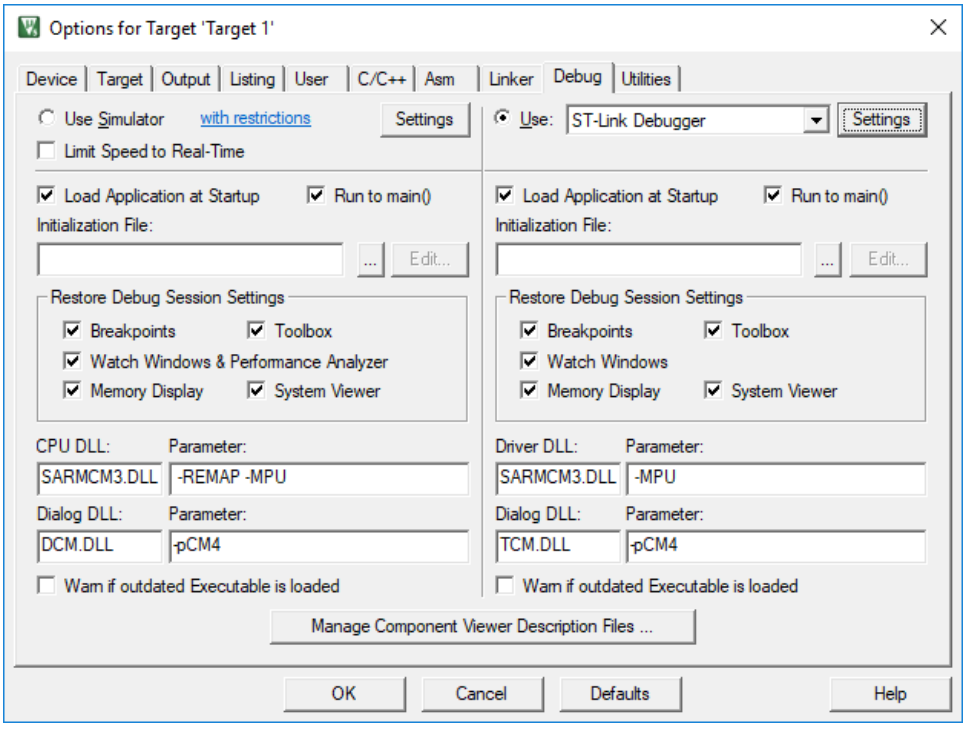
d. After clicking “OK” the **Manage Run-Time Enivronment** page appears. This is where we specify that the project will use the FreeRTOS operating system. Make the selections shown on the screenshot below. Notice that we are selecting the NUCLEO board, specifying the use of FreeRTOS, enabling timers, setting up the heap, and instructing the OS to begin on system startup. Click “OK” when you are done. **Note: These settings may vary slightly depending on the versions of Keil and FreeRTOS that you are using. The goal is to get all of the shaded boxes to be green, rather than yellow or red.**



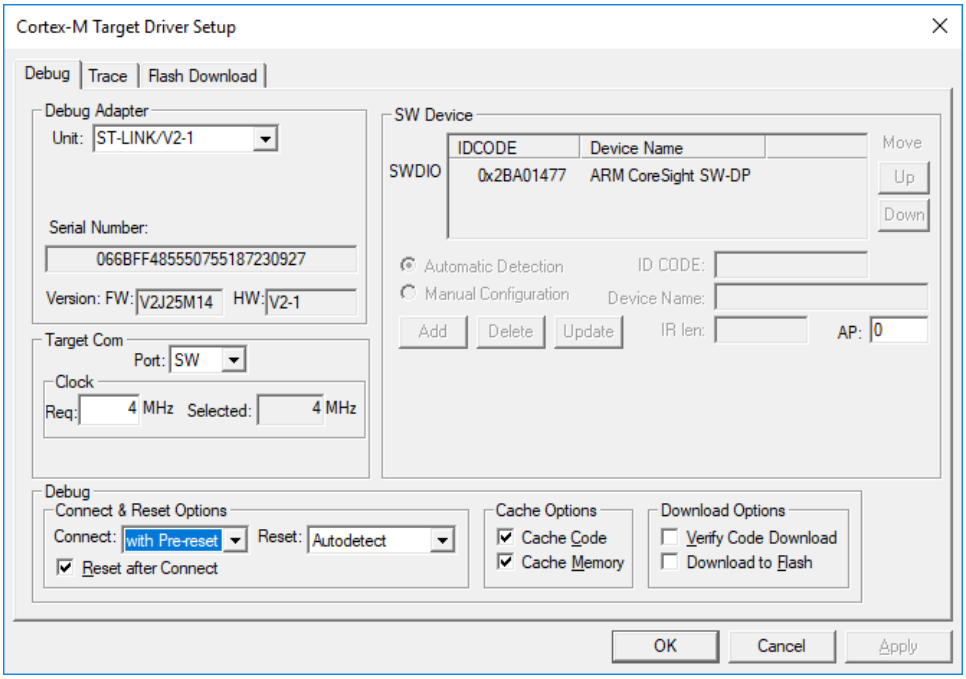
e. Click **Project->Options for Target 'Target 1'**, select the **Linker** tab, and make sure **Use Memory Layout from Target Dialog** is selected. The linker will fail otherwise. Also, paste the string "--diag_suppress=L6314W" into the **Misc controls** text box as shown.



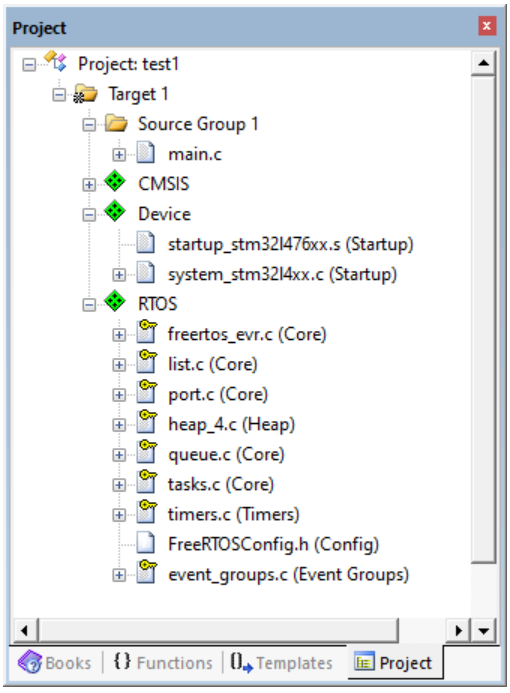
f. Still in the Options dialog box, click on the **Debug** tab, select **Use** in the right-hand column, then select **ST-Link Debugger** from the drop-down menu.



g. Click on **Settings** in the top-right corner, select the **Debug** tab, and select **with Pre-reset** in the **Connect** drop-down menu of the **Connect & Reset Options** section (bottom left). Then click **OK** until all pop-ups are closed.



h. Notice the Project pane on the left side of **µVision**. Pay special attention to the included files under Device and RTOS. Look through these files and familiarize yourself with the contents.



i. Right-click on “Source Group 1”, select “Add new item to group…”, and create a new C File named main.c.

3. Using the concepts discussed in class and found in the FreeRTOS documentation (specifically Chapter 2 of the FreeRTOS Reference Manual), add appropriate functionality to the newly-created main.c file to do the following:
   a. Configure appropriate GPIO pins.
   b. Create a task to control the LED. This task should simply turn the LED on or off based on input from the push button.
   c. Create a second task to monitor the push button. Each time the button is pressed, the LED should toggle between on and off. Use a global variable to share information between the two tasks.
   d. Start the FreeRTOS scheduler.

4. Pass-off
   a. Demonstrate the working system to the instructor, either in-person, via Zoom, or via a recorded video (emailed to the instructor). Make sure your pass-off shows all of the requirements listed in Procedure 3.